

CS3006 Assignment 2 — Performance Analysis on a Multi-Core CPU

Name: Muhammad Qudama Roll Number: 23L-0852 Date: 20 September 2026

1. Machine Declaration

This assignment was run inside a VMware Workstation virtual machine on a Windows host.

Field	Value
CPU model	Intel Core i7-8650U @ 1.90 GHz (Lenovo ThinkPad T480s)
Physical cores (C)	4
Hardware threads (T)	8
SMT present?	Yes on host silicon; NOT visible to guest
Base / max clock	1.90 GHz / 4.20 GHz
Widest SIMD (W)	AVX2, 8-wide single precision
RAM	8 GB allocated to VM; host 16 GB LPDDR3-2133 dual channel, soldered
Machine type	VMware Workstation VM, Windows host, own laptop
OS / kernel	Ubuntu 26.04 “resolute”, kernel 7.0.0-31-generic
GCC	15.2.0
ISPC	1.31.0 (LLVM 23.0.0)
Power state	Plugged in, Windows “Best performance”

Derived constants: $C = 4$, $T = 8$, $W = 8$. Program 3 part 2 ceiling = $C \times W = 32$.

Note on virtualised topology

The guest reports Thread(s) per core: 1, Core(s) per socket: 2, Socket(s): 4, and /sys/devices/system/cpu/cpu0 returns 0.

The guest operating system does not see the real shape of the CPU. It reports 4 sockets with 2 cores each and 1 thread per core, and `thread_siblings_list` returns only 0, meaning it believes CPU 0 has no sibling. The real hardware is a single Intel Core i7-8650U with 4 physical cores and 8 hardware threads, so the 8 vCPUs the VM sees are really 4 cores each running 2 hyperthreads.

Because of this I use $C = 4$ and not $C = 8$. Using 8 would give a Program 3 ceiling of 64, which the hardware cannot deliver, since two hyperthreads on one core share the same floating-point units. The correct ceiling is $4 \times 8 = 32$.

For Program 1 this means I can still observe the effect of hyper-threading, but the operating system cannot make SMT-aware scheduling decisions because it does not know which logical CPUs are siblings. So the measured drop in efficiency past 4 threads is real, but the guest cannot identify the cause on its own.

Measurement protocol

I followed the Section 5.2 protocol: I followed the protocol in Section 5.2. I closed all other applications on both the Windows host and inside the virtual machine, kept the laptop plugged in with Windows set to Best performance, and checked before each measurement run that memory was free and that swap usage was 0 B, so no timing was affected by paging. Each configuration was run at least 5 times and I report the minimum,

and I also report the minimum and maximum for at least one configuration in each program so the spread is visible. I left the machine to settle between long runs.

One part of the protocol I could not follow. The command `cpupower frequency-set -g performance` does not work inside a VMware guest, because the guest has no control over the physical CPU frequency. Clock control belongs to the Windows host, so I set the host power plan to Best performance instead. This is the closest equivalent available to me and I mention it for honesty.

2. Build Notes

- Added `#include <cstring>` to `prog1_mandelbrot_threads/main.cpp`
- Added `#include <cstdlib>` to `prog1_mandelbrot_threads/mandelbrotThread.cpp`
- ISPC v1.31.0 as required by the addendum (not Stanford's v1.28.1)
- Programs 2-6 built cleanly as shipped
- No additional fixes were needed despite GCC 15.2 being newer than the GCC 13 the addendum targets
- `prog6_kmeans/requirements.txt` pin ignored; installed current matplotlib 3.11.2 and scikit-learn 1.9.1 per addendum Section 7.4

3. Program 1 — Mandelbrot with Threads

3.1 Block decomposition (first attempt)

Threads	Speedup	Min per-thread (ms)	Max per-thread (ms)	Max/Min
2	1.90x	220.989	223.014	1.01
3	1.54x	91.578	277.214	3.03
4	2.17x	51.217	223.934	4.37
8	3.38x	10.521	127.664	12.61

View 2, 4 threads: 2.30x, min 55.281 ms, max 115.930 ms (ratio 2.10) Serial baseline: view 1 ~424-451 ms, view 2 ~252 ms

3.2 Why speedup is not linear

The speedup is not linear because the threads do not get equal amounts of work. In view 1 the expensive pixels are the ones inside the Mandelbrot set, because those points never escape and the loop runs the full number of iterations. These points sit in a band across the middle of the image. The block decomposition gives each thread one continuous strip of rows, so whichever thread owns the middle gets far more work than the others.

With 2 threads the split falls at row 600, which cuts the expensive band in half. Both threads get a similar amount of work, so the speedup is 1.90x, close to ideal. With 3 threads, thread 1 owns rows 400 to 799, which covers the whole expensive band. It takes 277 ms while threads 0 and 2 finish in about 95 ms and then sit idle. The total time is set by the slowest thread, not the average, so the speedup drops to 1.54x. This means 3 threads is actually slower than 2 threads, which is the clearest evidence that the problem is imbalance and not a lack of parallelism.

At 4 threads the band is split between threads 1 and 2, and threads 0 and 3 finish early. At 8 threads the per-thread times form a smooth gradient from 10 ms to 127 ms, a ratio of 12.6 to 1. View 2 shows the same flaw with the hot region in a different place: there thread 0 is the slow one at 112 ms while the others finish in about 60 ms.

3.3 Row-cyclic decomposition (the fix)

Thread i computes rows i , $i+\text{numThreads}$, $i+2*\text{numThreads}$, $\dots$

Threads	Speedup	Min (ms)	Max (ms)	Max/Min
2	1.96x	216.450	216.432	1.00
3	2.69x	157.822	161.426	1.02
4	3.24x	135.091	138.278	1.02
8	5.18x	70.177	85.868	1.21

View 2, 4 threads: 3.32x, min 69.470 ms, max 76.799 ms (ratio 1.11)

Before vs after: 2t 1.90 -> 1.96, 3t 1.54 -> 2.69, 4t 2.17 -> 3.24, 8t 3.38 -> 5.18, view2 4t 2.30 -> 3.32

Interleaving works because neighbouring rows cost almost the same amount of time, while rows far apart can cost very different amounts. By giving thread i the rows i , $i+\text{numThreads}$, $i+2*\text{numThreads}$ and so on, each thread ends up with a sample of rows taken evenly from across the whole image. Every thread therefore receives a similar mixture of cheap rows and expensive rows, and no single thread can end up owning the whole expensive band.

The evidence is the Max/Min column. At 4 threads the block version had a ratio of 4.37, meaning the slowest thread took more than four times as long as the fastest. The cyclic version has a ratio of 1.02, so all four threads finish within about 3 ms of each other out of roughly 140 ms. The 3-thread anomaly also disappears: speedup rises from 1.54x to 2.69x, and speedup now increases with every extra thread as it should.

3.4 Speedup at T threads vs ideal T

Achieved **5.18x** at $T = 8$ against an ideal of 8x (65% efficiency).

The achieved 5.18x against an ideal 8x is 65 percent efficiency. There are three separate reasons for the gap and they should not be confused with each other.

First and most importantly, the machine only has 4 physical cores. Running 8 threads means 4 cores each running 2 hyperthreads, and the two siblings on a core share the same floating-point execution units. Mandelbrot is dense floating-point work that already keeps those units busy, so the second thread on a core finds very little spare capacity. My own numbers show this directly: going from 4 threads to 8 threads doubled the thread count but only improved speedup from 3.24x to 5.18x, a factor of 1.60. That is consistent with 4 real cores plus a modest hyper-threading bonus of the usual 20 to 30 percent.

Second, a small amount of imbalance remains. The spread between fastest and slowest thread is 1.02 at 4 threads but widens to 1.21 at 8 threads, because with a stride of 8 each thread gets only 150 rows and the sampling across the image is coarser.

Third, the measurement platform depresses the number. The program runs in a VMware guest on a 15 W laptop chip. The serial baseline runs on a single core and can use the high single-core turbo of 4.2 GHz, while the 8-thread run pushes all cores and the sustained all-core clock is much lower. Since speedup is the ratio of those two times, a lower parallel clock mechanically reduces the measured speedup even if the parallel code is perfect.

3.5 Part 5 — running with $2T = 16$ threads

16 threads: **5.15x**, versus 5.18x at 8 threads. No improvement. Per-thread spread at 16t is ~34-87 ms (ratio ~2.5) vs 1.21 at 8t.

Running 16 threads gives 5.15x, which is the same as the 5.18x from 8 threads once measurement noise is taken into account. Adding more software threads than the machine has hardware threads does not add any execution resources. It only adds threads waiting for a turn, so the operating system time-slices 16 runnable threads across 8 logical CPUs and the same total work takes about the same wall-clock time.

My per-thread numbers confirm this. At 16 threads each thread handles 75 rows instead of 150, so half the work, yet typical per-thread times are 60 to 80 ms compared with 70 to 86 ms at 8 threads. Half the work taking almost the same time per thread shows each thread was running at roughly half speed.

The spread also widens, from a ratio of 1.21 at 8 threads to about 2.5 at 16 threads. This is not load imbalance. Every thread has exactly 75 rows of comparable cost, so the work really is equal. It is scheduling jitter: a thread that gets descheduled part way through its rows records a longer wall-clock time even though it did identical work. Imbalance and scheduling noise can look similar in the numbers but they have different causes.

There is also no slowdown, which is worth noting. Oversubscription normally costs context switches and cache pressure, but Mandelbrot rows run for a long time and have a very small working set, so switches are rare relative to the work and the caches stay warm. On a workload with a larger memory footprint, 16 threads would have been measurably slower than 8.

The overall conclusion is that useful parallelism is limited by the number of hardware threads, not by how many threads the program asks for.

4. Program 2 — Vector Ininsics

Workload size $N = 10000$.

VECTOR_WIDTH	Utilization	Total vector instructions
2	77.9%	167,727
4	70.7%	97,075
8	67.0%	52,877
16	65.2%	27,592

CLAMPED EXPONENT and ARRAY SUM both passed at every width.

Utilisation goes down as the vector gets wider. This happens because of the `while (count > 0)` loop. All the lanes in one vector must keep looping until the lane with the largest exponent has finished. Any lane that has already finished is masked off, but it still occupies its place in the vector and does no useful work.

The exponents are random numbers from 0 to 9. With a width of 2, the two exponents in a group are often close to each other, so little time is wasted. With a width of 16 it is very likely that at least one of the 16 lanes holds a 9, which forces all 16 lanes to keep looping while most of them are already done. So the wider the vector, the more likely there is one slow lane holding everything up, and the more lanes are wasted.

This is the same underlying problem as the load imbalance in Program 1, but happening inside a single vector instruction instead of across threads. The general lesson is that SIMD works best when every lane follows the same path, and loses efficiency whenever the lanes diverge.

5. Program 3 — Mandelbrot with ISPC

5.1 Part 1: SIMD only. Ceiling = $W = 8$

View 1: **4.59x** View 2: **4.12x**

The ceiling for SIMD alone is $W = 8$, because AVX2 processes 8 single-precision values per instruction. I measured 4.59x on view 1 and 4.12x on view 2, so about 57 and 51 percent of the ceiling.

The main reason for falling short is divergence inside `mandel()`. The 8 pixels being processed together escape the loop at different iteration counts. A lane that escapes early is masked off, but it still occupies its slot in

the gang and the gang keeps iterating until the slowest lane finishes. So the gang costs as much as its worst pixel, and the wasted lanes never do useful work. This is the same effect measured directly in Program 2.

A second and smaller reason is in the build flags. The Makefile passes `--opt=disable-fma` to ISPC and `-ffp-contract=off` to GCC, which stops the compiler from fusing multiply and add into a single instruction. This is done so the serial and ISPC results are bit-identical and verification passes, but it costs some floating-point throughput that the hardware could otherwise provide.

5.2 Part 2: with tasks. Ceiling = $C \times W = 4 \times 8 = 32$

Task count sweep, view 1, 3 runs each (best of 3 shown):

Tasks	Best speedup from task ISPC
2	10.22x
4	13.12x
8	17.77x
16	24.70x

Final 5 runs at NUM_TASKS = 16: - ISPC only: 5.65 / 5.54 / 5.04 / 5.44 / 4.62 — best **5.65x** - Task ISPC: 30.09 / 25.79 / 27.22 / 22.15 / 24.76 — best **30.09x**, spread 22.15-30.09

Achieved 30.09x of the 32x ceiling = 94%.

The ceiling with tasks is $C \times W = 4 \times 8 = 32$. I achieved 30.09x, which is 94 percent of the ceiling. The remaining 6 percent is accounted for by the same lane divergence described above, the cost of launching and scheduling tasks, and the lower sustained all-core clock on this laptop chip compared with the single-core clock used by the serial baseline.

The more interesting result is that 16 tasks beats 8 tasks, even though the machine has only 8 hardware threads. With exactly 8 tasks, each thread gets one task and keeps it to the end, so a thread that receives easy rows finishes early and then sits idle. With 16 tasks the work is cut into smaller pieces and handed out dynamically, so a thread that finishes an easy piece immediately picks up another. This keeps every thread busy until near the end of the whole computation.

This is the same load-imbalance problem I solved by hand in Program 1 using row-cyclic decomposition. Here I did not have to design the distribution myself: creating more tasks than threads lets the runtime balance the load for me. Over-decomposition is a simpler and more general fix than hand-tuning the decomposition.

Methodology note: My first sweep of task counts was wrong and I want to record how I found out. I ran the binary without the `--tasks` flag, which means it never executed the task version at all. Every task count produced almost exactly the same time, around 51 ms, and the speedups moved up and down at random between 4.46x and 6.19x with no trend.

I noticed the problem because the numbers made no sense. Task count should change the result, and the values were not increasing with more tasks. Checking `./mandelbrot_ispc --help` showed a `-t / --tasks` option, and checking `main.cpp` confirmed that `mandelbrot_ispc_withtasks` is only called when that flag is given. After re-running the sweep with `--tasks`, the results became clean and monotonic: 10.22x, 13.12x, 17.77x and 24.70x for 2, 4, 8 and 16 tasks.

I also found a related bug in my own setup. I had initially tested 32 and 64 tasks, but the image is 1200 rows tall and `height / 64` truncates to 18, so 64 tasks cover only 1152 rows and the last 48 rows are never computed. I restricted the sweep to task counts that divide 1200 exactly.

6. Program 4 — sqrt

Input	Serial (ms)	ISPC (ms)	Task ISPC (ms)	ISPC speedup	Task speedup
Random (starter)	1141.7	314.0	52.1	3.64x	21.92x
Max-speedup input	2754.3	422.1	94.4	6.53x	29.17x
Min-speedup input	269.7	322.4	59.8	0.84x	4.51x

Max input: every element set to 2.998f **Min input:** `values[i] = (i % 8 == 0) ? 2.998f : 1.0f`

For the maximum-speedup input I set every element to 2.998f, the largest value in the original range. Every element then needs the same large number of Newton iterations to converge. Because all 8 lanes in a gang follow exactly the same path and finish at the same time, no lane is ever masked off waiting for another, and the vector unit is fully used. This gave 6.53x against the $W = 8$ ceiling, compared with 3.64x for the random starter input.

For the minimum-speedup input I used `values[i] = (i % 8 == 0) ? 2.998f : 1.0f`. The modulus of 8 matches the gang width, so exactly one element in every 8-wide group is the expensive value while the other seven are 1.0f and converge after a single iteration. Those seven lanes finish immediately but must wait, masked off, while the gang keeps iterating for the one slow lane. The vector therefore does 8 lanes of work to produce 1 lane of useful result.

This is the only case where ISPC is slower than the serial code, at 0.84x. The serial version simply skips the cheap elements after one iteration and moves on, so it finishes in 270 ms, while the vector version cannot skip them and takes 322 ms. This shows that SIMD is not automatically a win: when the data causes heavy divergence, vectorisation can be a net loss.

The task version still gives a large speedup in both cases, 29.17x and 4.51x. This is because multi-core parallelism does not depend on the lanes agreeing with each other. Splitting work across cores helps regardless of divergence, so the two forms of parallelism fail in different ways and for different reasons.

7. Program 5 — saxpy

5 runs (ISPC): 24.910 / 14.557 / 14.892 / 16.786 / 14.702 ms Best: **14.557 ms = 20.473 GB/s**. Spread 11.96 - 20.47 GB/s. Tasks: 0.99x and 0.82x — no benefit.

Theoretical peak bandwidth: LPDDR3-2133, dual channel, soldered (ThinkPad T480s) 2 channels x 8 bytes x 2133 MT/s = **34.1 GB/s**

Measured 20.5 GB/s = 60% of theoretical peak.

saxpy computes `result = a*X + Y`. For every element it reads two floats and writes one, and the harness counts 16 bytes of traffic for 2 floating-point operations. That ratio of 8 bytes moved per operation is extremely low, which makes this kernel completely memory bound. The processor spends nearly all its time waiting for data from DRAM rather than computing.

The theoretical peak bandwidth of this machine is 34.1 GB/s, from LPDDR3-2133 in dual channel: 2 channels x 8 bytes x 2133 MT/s. I measured a best of 20.5 GB/s, which is 60 percent of that figure. This is a normal result. Theoretical peak assumes perfectly sequential access with no refresh cycles, no row activation delays and no other traffic, none of which is achievable in practice. Real applications typically reach 60 to 75 percent of theoretical peak, so 60 percent is reasonable, and being at the low end of that range is expected on a virtual machine where the host is also using memory.

Using tasks gave 0.99x and 0.82x, so no benefit and sometimes a loss. This is the expected outcome for a memory-bound kernel. A single core issuing streaming reads can already saturate most of the available memory bandwidth, so adding more cores does not add any more bandwidth. The bottleneck is the DRAM interface, not the number of processors. Extra threads only add synchronisation cost and compete for the same limited bandwidth, which explains the 0.82x result on one run.

The lesson is that adding parallelism only helps if the resource you are short of is compute. Here the shortage is memory bandwidth, and no amount of extra cores can change that.

8. Program 6 — K-Means

md5sum of data.dat: 3a25f24193f4fdca82ee4cb2737fd5bb **Size:** 804,002,420 bytes (766.8 MiB) — matches the addendum

8.1 Profile of the serial code (101 iterations)

Function	Time (ms)	Fraction
computeAssignments	43,733.009	0.6902
computeCentroids	6,675.950	0.1054
computeCost	12,953.382	0.2044
Total	63,362.341	

f = 0.6902, hotspot is `computeAssignments`.

8.2 Amdahl ceiling

$$\begin{aligned}
 S_{\max} &= 1 / ((1 - f) + f/T) \\
 &= 1 / ((1 - 0.6902) + 0.6902/8) \\
 &= 1 / (0.3098 + 0.0863) \\
 &= 1 / 0.3961 \\
 &= 2.525x
 \end{aligned}$$

Requirement: $0.80 \times 2.525 = \mathbf{2.02x}$

8.3 Achieved

Serial baseline: 63,556 ms Parallel, 5 runs: 24,448 / 29,973 / 28,584 / 33,305 / 32,968 ms Minimum: **24,448**
ms Speedup: $63,556 / 24,448 = \mathbf{2.60x}$ Spread: 24,448 - 33,305 ms (36%)

8.4 Profiling narrative

I started by instrumenting the three functions in the main loop with `CycleTimer`, accumulating the time spent in each across all iterations. Over 101 iterations the profile showed `computeAssignments` taking 43,733 ms of 63,362 ms total, which is a fraction *f* of 0.6902. `computeCost` took 0.2044 and `computeCentroids` took 0.1054. This told me `computeAssignments` was the hotspot and the only part worth parallelising first.

Looking at the code, the original `computeAssignments` loops over centroids on the outside and data points on the inside, and keeps a `minDist` array of one million doubles. That structure is difficult to thread safely, because every centroid iteration writes to the same shared arrays for every point. It is also inefficient: for each of the *K* centroids it walks the entire 800 MB data array from start to finish, so the data is read *K* times in total and almost nothing stays in cache between passes.

I therefore made two changes. First I swapped the loop order so that data points are on the outside and centroids on the inside. Each point now loads its 100 doubles once and compares them against all 3 centroids

while that data is still in cache, and the `minDist` array is no longer needed because the best distance is tracked in a local variable. Second, because each data point is now completely independent of every other, I split the M points into 8 contiguous ranges and gave one range to each thread. No locking is needed, since each thread writes only to its own section of `clusterAssignments`.

The result was a drop from 63,556 ms to a best of 24,448 ms, a speedup of 2.60x. The profile after the change shows `computeAssignments` fell from 43,733 ms to 8,407 ms and is no longer the largest component. `computeCost`, which I left serial, now takes 0.4670 of the remaining time and would be the natural next target.

8.5 Exceeding the Amdahl ceiling

2.60x is **above** the computed 2.525x ceiling.

The measured speedup of 2.60x is higher than the calculated ceiling of 2.525x, which looks impossible at first. It is not an error, and the reason is that Amdahl's law does not apply cleanly to what I actually changed.

Amdahl's law assumes the work inside the hotspot stays the same and is only divided among more threads. I made two changes at once. The first was threading, which is what Amdahl's law describes. The second was swapping the loop order, which reduced the amount of work the hotspot has to do. The original code read all 800 MB of data K times, once per centroid, and had very poor cache behaviour. My version reads each point's data once and reuses it for all K comparisons while it is still in cache.

Because the hotspot became cheaper as well as parallel, the ceiling computed from the original profile no longer bounds the result. If I had only added threads and left the loop order alone, the speedup would have been capped at 2.525x as the formula predicts. This is a useful reminder that Amdahl's law bounds parallelisation, not optimisation, and that improving the serial algorithm is sometimes the more valuable change.

8.6 Karp-Flatt check

$$\begin{aligned} e &= (1/S - 1/T) / (1 - 1/T) \\ &= (1/2.60 - 0.125) / 0.875 \\ &= (0.3846 - 0.125) / 0.875 \\ &= 0.297 \end{aligned}$$

Measured serial fraction $(1 - f) = 0.3098$.

The Karp-Flatt metric gives an experimentally determined serial fraction of 0.297, while the profile I measured directly gives a serial fraction of 0.3098. The two agree to within about 4 percent.

This agreement is a useful cross-check. The Karp-Flatt value is derived only from the measured speedup and the thread count, and knows nothing about the profile. The 0.3098 comes from timing the three functions individually and knows nothing about the speedup. Two independent routes landing on nearly the same number gives reasonable confidence that the profile is accurate and that the speedup is not an artefact of measurement error.

The small remaining difference is in the direction I would expect. Karp-Flatt reports slightly less serial work than the profile does, which is consistent with the rewritten hotspot being cheaper than the original profile assumed.

8.7 Relative to the Stanford 2.1x target

The Stanford target of 2.1x sits inside my machine's ceiling of 2.525x, so unlike a dual-core machine I was able to reach it. My achieved 2.60x exceeds both the Stanford figure and my own calculated ceiling.

It is worth being precise about why 2.1x was reachable here. My hardware thread count of $T = 8$ matches the 4-core, 8-thread topology the Stanford handout assumes, so the arithmetic works out similarly. If this

machine had only 2 cores and 4 threads, the ceiling with the same $f = 0.6902$ would be $1 / (0.3098 + 0.1726) = 2.07x$, and $2.1x$ would have been just out of reach no matter how good the code was.

8.8 Plots

`plots/start.png` and `plots/end.png` are included.

Both plots are included in `plots/`. In `start.png` the three initial centroids sit almost on top of each other near the middle of the point cloud, because `initCentroids` deliberately places them close together. In `end.png` the three centroids have separated and each sits inside its own group of points, which confirms the algorithm converged correctly and that my parallel version produces the same clustering as the serial one.

As the addendum warns, these images project 100-dimensional points down to 2 dimensions using PCA, so individual points can appear to be assigned to the wrong centroid while being correctly assigned in the real 100-dimensional space. What the plot actually demonstrates is that three coherent clusters formed and that each centroid lies inside one of them, not that every individual point is on the correct side of a visible boundary.

9. Extra Credit

Not attempted.

10. Measurement Honesty Statement

Every number in this write-up was measured by me on the machine declared in Section 1. No timings were copied from the Stanford handout, from classmates, or from any other machine.

The main limitation of these measurements is the platform. The programs ran inside a VMware virtual machine on a 15 W laptop processor, which is a noisy measurement instrument compared with a dedicated lab machine. The spread in my repeated runs shows this clearly: Program 6 varied from 24,448 ms to 33,305 ms, a spread of 36 percent, and Program 3 with tasks varied from 22.15x to 30.09x across five runs. Following the protocol and reporting the minimum of five runs reduces the effect of this noise but does not remove it. Where a result depends on a small difference between two numbers, it should be read with that spread in mind.